초보자용 안내서 · Windows / VS Code 기준

Git 개념 이해와 기본 사용법

파일의 변경을 기록하고, 이전 상태를 확인하며, GitHub와 주고받는 방법을 차근차근 익힌다.

먼저 기억할 흐름: **수정 → add → commit → push** · 최신 내용 받기: **pull**

한 문장으로 말하면

Git은 내 컴퓨터의 프로젝트 변경 기록을 관리하는 도구이고, GitHub는 Git 저장소를 인터넷에서 보관하고 공유하는 서비스다.

이 안내서에서 배울 내용

- Git과 GitHub, 왜 사용하는가?
- Git의 세 작업 공간
- 처음 한 번 설정하기
- 내 변경사항 기록하고 올리기
- 저장소 복사하고 최신 내용 받기
- 브랜치와 병합 맛보기
- 기본 명령어와 문제 해결

1. Git과 GitHub, 왜 사용하는가?

문서를 수정할 때마다 `최종`, `최종_진짜`, `최종_진짜2`처럼 파일을 복사해 두면 어떤 파일이 최신인지 헷갈릴 수 있다. Git은 변경한 내용을 기록해 두어 **무엇을 언제 바꿨는지 확인**하고, 필요할 때 이전 버전을 살펴볼 수 있게 돕는다.

이름	쉬운 설명	비유
Git	내 컴퓨터의 프로젝트 변경 이력을 기록하고 관리하는 버전 관리 도구	프로젝트의 저장·기록 장치
GitHub	Git 저장소를 온라인에 보관하고 다른 사람과 공유하는 서비스	인터넷에 있는 프로젝트 보관함
저장소 (Repository)	프로젝트 파일과 Git의 변경 이력이 함께 관리되는 공간	프로젝트 전용 기록장

> Git은 인터넷 연결 없이도 내 컴퓨터에서 기록할 수 있다. GitHub에 올리거나 내려받을 때는 인터넷 연결이 필요하다.

2. Git의 세 작업 공간 이해하기

Git은 파일을 한 번에 무조건 기록하지 않는다. 먼저 작업하고, 기록할 내용을 고르고, 그 선택을 하나의 버전으로 저장한다.

```
작업 디렉터리 (Working Directory)  수정 중인 파일
  ↓ git add
스테이징 영역 (Staging Area)  이번 기록에 넣을 변경사항을 골라 둔 곳
  ↓ git commit
로컬 저장소 (Local Repository)  내 컴퓨터에 저장된 버전 기록
  ↓ git push / ↑ git pull
원격 저장소 (Remote Repository)  GitHub에 있는 저장소
```

- **작업 디렉터리**: 파일을 열어 수정하고 저장하는 곳이다.
- **스테이징 영역**: 이번 버전에 포함할 파일을 잠시 준비하는 곳이다. `git add`가 이 단계다.
- **로컬 저장소**: 준비한 변경사항을 커밋으로 기록하는 내 컴퓨터 안의 저장소다.
- **원격 저장소**: GitHub 등에 있는 온라인 저장소다. `push`로 올리고 `pull`로 가져온다.

> **중요:** `commit`은 내 컴퓨터에 기록하는 것이고, `push`는 그 기록을 GitHub에 올리는 것이다. 커밋만 했다고 GitHub에 자동으로 올라가지는 않는다.

3. 처음 한 번 설정하기

준비

- Git 설치
- Visual Studio Code 설치
- GitHub에 올릴 프로젝트라면 GitHub 계정과 저장소

VS Code에서 **Terminal(터미널) > New Terminal(새 터미널)**을 연다. 사용자 이름과 이메일은 커밋 기록에 작성자로 표시된다. 본인 정보를 따옴표 안에 넣어 한 번 설정한다.

```
git config --global user.name "내 이름"
git config --global user.email "내 이메일 주소"
```

예: 이메일은 GitHub 계정에 등록한 주소를 사용할 수 있다. 설정이 잘 되었는지 확인하려면 아래 명령을 실행한다.

```
git config --global --list
```

> 이 설정은 컴퓨터의 Git 커밋 작성자 이름과 이메일을 지정한다. 비밀번호나 인증 토큰은 이곳에 입력하지 않는다.

4. 내 변경사항을 기록하고 GitHub에 올리기

이미 Git 저장소로 만든 프로젝트에서 파일을 수정한 뒤, 아래 순서로 진행한다.

1. **상태 확인** — 어떤 파일이 바뀌었는지 살펴본다.

   ```
   git status
   ```

2. **기록할 파일 고르기** — 지정한 파일만 준비하려면 파일 이름을 쓴다.

   ```
   git add README.md
   ```

   변경한 파일을 모두 준비하려면 다음처럼 쓸 수 있다.

   ```
   git add .
   ```

처음에는 `git add .` 보다 파일별로 추가하고, 불필요한 파일(비밀번호·개인정보·큰 임시파일 등)이 포함되지 않았는지 확인하는 습관이 안전하다.

3. **준비된 내용 확인**

```
git status
```

준비한 파일은 보통 "Changes to be committed" 영역에서 확인할 수 있다.

4. **커밋으로 기록** — 이번 변경사항을 설명하는 짧은 메시지를 작성한다.

```
git commit -m "실습 안내 문서 수정"
```

5. **GitHub로 올리기** — 저장소가 GitHub 원격 저장소와 연결되어 있을 때 실행한다.

```
git push
```

저장소 설정에 따라 처음에는 `git push -u origin main` 이 필요할 수 있다. 안내된 브랜치 이름이 `main` 이 아니라면 해당 이름을 사용한다.

> **기억할 순서**
> `git status` → `git add` → `git commit` → `git push`
> 작업한 내용을 확인하고, 선택하고, 내 컴퓨터에 기록한 뒤, 원격 저장소에 공유한다.

좋은 커밋 메시지 예시

- `실습 회로도 설명 추가`
- `fix: LED 예제의 핀 번호 수정`
- `docs: 설치 안내 보완`

메시지는 "무엇을 바꿨는지" 알아볼 수 있게 간결하게 적는다. `수정` , `최종` 처럼 내용이 드러나지 않는 메시지는 피하는 것이 좋다.

5. 내 컴퓨터의 새 프로젝트를 처음 GitHub에 올리기

아래는 **내가 만든 새 프로젝트**를 내 GitHub 저장소에 처음 올리는 경우에만 필요하다. 수업 저장소를 클론해서 사용하는 학생은 이 절차 대신 Clone과 Pull을 사용한다.

1. GitHub에서 새 저장소를 만든다. 처음 연습할 때는 README 없이 만든 **빈 저장소**가 가장 간단하다.
2. VS Code에서 프로젝트 폴더를 열고 **Terminal > New Terminal**을 선택한다.
3. 이 폴더를 Git 저장소로 한 번 초기화한다.

```
git init
```

4. 기록할 파일을 준비하고 커밋한다.

```
git add .
git status
git commit -m "첫 버전 저장"
```

5. 새 저장소의 기본 브랜치를 `main`으로 지정한다.

```
git branch -M main
```

6. GitHub 저장소의 HTTPS 주소를 원격 저장소로 연결한다. 주소는 본인의 저장소 페이지에서 복사한다.

```
git remote add origin https://github.com/사용자이름/저장소이름.git
```

7. 내 컴퓨터의 첫 커밋을 GitHub에 올린다.

```
git push -u origin main
```

> **주의:** 이미 README나 파일이 들어 있는 GitHub 저장소와 연결하면 첫 Push에서 서로 다른 기록 때문에 추가 처리가 필요할 수 있다. 기존 저장소가 있다면 새로 `git init`하기보다 먼저 `git clone`으로 가져오는 방법을 권장한다. `git init`도 같은 폴더에서 반복할 필요가 없다.

6. 저장소 복사하고 최신 내용 받기

Clone — 처음 한 번 프로젝트 가져오기

`clone`은 GitHub 저장소와 파일, 변경 기록을 내 컴퓨터에 복사하는 작업이다. VS Code에서 **Ctrl + Shift + P**를 누르고 `Git: Clone`을 선택한 다음, GitHub 저장소 주소를 붙여넣으면 된다.

```
git clone https://github.com/사용자이름/저장소이름.git
```

복사한 폴더를 VS Code에서 열어 작업한다. 이미 클론한 프로젝트는 다시 클론할 필요 없이 그 폴더에서 계속 작업하면 된다.

Pull — 다른 사람이 올린 최신 내용 받기

작업을 시작하기 전에 원격 저장소의 최신 변경사항을 받아오려면, 저장소 폴더에서 실행한다.

```
git pull
```

`git pull`은 원격 저장소의 변경을 가져와 현재 브랜치에 반영한다. 선생님이나 팀원이 업데이트한 수업 파일을 받을 때 사용한다.

Fetch와 Pull의 차이

- `git fetch` : 원격의 새 기록을 확인해 가져오지만, 내 작업 파일에 바로 합치지는 않는다.
- `git pull` : 원격의 새 기록을 가져오고 현재 브랜치에 반영한다.

> 같은 파일의 같은 부분을 나와 다른 사람이 서로 다르게 수정하면 충돌(conflict)이 날 수 있다. 당황해서 파일을 지우거나 강제로 덮어쓰지 말고, 변경된 내용을 확인한 뒤 담당자와 함께 해결한다.

7. 브랜치와 병합 맛보기

브랜치(Branch)는 기존 작업을 그대로 둔 채 새 작업을 따로 진행하는 갈래다. 실험이나 기능 개발을 별도 브랜치에서 진행하면 안정된 기본 브랜치를 보호할 수 있다.

```
git branch                  # 브랜치 목록 확인
git switch -c feature-docs  # 새 브랜치를 만들고 이동
git switch main             # main 브랜치로 이동
```

작업을 마친 브랜치의 변경사항을 현재 브랜치에 합치는 것을 **병합(Merge)**이라고 한다. 처음에는 선생님 또는 팀의 안내에 따라 진행한다.

```
git switch main
git merge feature-docs
```

> 브랜치 이름은 예시이다. 기본 브랜치가 `main` 대신 `master` 또는 다른 이름일 수도 있으니 저장소에서 확인한다.

8. 자주 쓰는 명령어

명령어	하는 일	언제 사용하는가?
`git status`	현재 변경·준비 상태 확인	무엇이 바뀌었는지 볼 때
`git add 파일명`	파일을 다음 커밋에 포함하도록 준비	기록할 파일을 고를 때
`git commit -m "메시지"`	준비한 변경사항을 로컬에 기록	작업 단위를 저장할 때
`git push`	내 커밋을 원격 저장소로 올림	GitHub에 작업을 공유할 때
`git pull`	원격 변경사항을 가져와 반영	다른 사람이 올린 최신 내용을 받을 때
`git log --oneline`	최근 커밋 기록을 간단히 표시	변경 이력을 확인할 때
`git diff`	아직 스테이징하지 않은 수정 내용 비교	무엇이 바뀌었는지 자세히 볼 때
`git diff --staged`	스테이징한 내용을 마지막 커밋과 비교	커밋 전에 준비 내용을 검토할 때

실수했을 때 안전하게 기억할 점

- `git status`를 먼저 실행해 현재 상태를 확인한다.
- 아직 커밋하지 않은 수정 내용을 버리는 명령은 복구하기 어려울 수 있다. 무엇을 지우는지 확실하지 않으면 실행하지 않는다.
- `git reset --hard`, `git push --force`는 작업이나 다른 사람의 기록을 잃게 할 수 있는 고급 명령이다. 초보자는 임의로 사용하지 말고 담당자에게 먼저 문의한다.
- 비밀번호, 토큰, 개인 정보가 들어간 파일은 커밋하거나 GitHub에 올리지 않는다. `.gitignore` 파일에 올리지 않을 파일 이름이나 패턴을 적을 수 있다.

9. 수업·실습에서 지킬 기본 습관

☐ 작업 전에 저장소 폴더가 맞는지 확인한다.

- [ ] 공동 작업이면 시작 전에 `git pull`로 최신 내용을 받는다.
- [ ] 커밋 전 `git status`로 올릴 파일을 확인한다.
- [ ] 작은 작업 단위로 이해하기 쉬운 커밋 메시지를 남긴다.
- [ ] 저장소 소유자나 수업 담당자의 허락 없이 공동 저장소에 Push하지 않는다.
- [ ] 충돌이나 오류가 나면 강제 명령을 사용하기 전에 도움을 요청한다.

> **초보자 핵심 요약**
> Git은 변경 기록을 관리하고, GitHub는 저장소를 온라인에서 공유한다.
> 내 작업은 `add → commit → push`, 최신 내용 받기는 `pull`, 현재 상태 확인은 `status` 이다.

참고

- 제공된 Google 문서의 'Git 사용법' 탭 — 개념과 실습 순서를 참고해 초보자용으로 재구성했다.
- Pro Git 한국어판
- Visual Studio Code 공식 문서: 소스 제어

초보자용 안내서 · Windows / VS Code 기준 · 명령어는 저장소 폴더 안의 터미널에서 실행한다. 화면과 메뉴 명칭은 프로그램 버전에 따라 조금 다를 수 있다.